

M06 — Elementary Data Structures

Sources: CLRS Part III intro, Ch. 10 (Elementary Data Structures) · Skiena Ch. 3 §§3.1–3.3, 3.5, 3.6, 3.8, 3.9

Big Idea

Every data structure is an answer to the question “*which operations must be fast?*” — and the answer always costs something elsewhere. The organizing distinction is **contiguous vs. linked**: arrays give you $O(1)$ indexing, zero space overhead, and excellent cache locality but a fixed size; linked structures give you $O(1)$ splicing and unbounded growth but pay a pointer per node and scatter your data across memory. Above that sits the **abstract data type** — dictionary, container, priority queue — which names the operations you need and lets you swap implementations underneath. The two tables in this module (dictionary operations across four array/list variants) are the single most useful thing here: they show concretely that *the fastest structure for operations A and B together is usually not the fastest for A or B alone*. And two ideas foreshadow later modules: **dynamic array doubling** is your first amortized argument ($2n$ total moves), and the $\Theta(n)$ **cost of finding a singly-linked predecessor** is why doubly linked lists exist.

What You Should Be Able To Do After This Chapter

- List the dynamic-set operations and say what each costs on an unsorted array, a sorted array, and each of the four linked-list variants — and **explain** each entry, including the starred clever ones.
- Explain the amortized $2n$ bound for dynamic-array doubling, and what guarantee it gives up.
- Implement stacks, queues (circular buffer), and deques with $O(1)$ operations, handling the empty/full ambiguity correctly.
- Implement a doubly linked list with a **sentinel** and explain what the sentinel buys and costs.
- Delete from a singly linked list in $O(1)$ given only a pointer to the node — and say what that trick breaks.
- Choose row-major vs column-major vs block layout, and compute the index

formulas.

- Represent a rooted tree with unbounded branching in $O(n)$ space (left-child, right-sibling).
- Implement a queue with two stacks and a stack with two queues, with correct amortized analysis.
- Recognize when the bottleneck is the *inner-loop data structure* and know the escalation ladder: array $\rightarrow$ BST $\rightarrow$ hash table $\rightarrow$ suffix tree $\rightarrow$ compressed suffix tree.

1. Dynamic sets and their operations

The framing [CLRS Part III intro, p.249]

*Whereas mathematical sets are unchanging, the sets manipulated by algorithms can grow, shrink, or otherwise change over time. We call such sets **dynamic**.*

Each element is an object with an identifying **key**, possibly **satellite data**, and possibly attributes used by the structure itself. Some structures assume keys come from a **totally ordered** set — which is what makes `MINIMUM`, `SUCCESSOR`, etc. meaningful.

The operation catalogue

OPERATION	KIND	MEANING
<code>SEARCH(S, k)</code>	query	pointer to an $x \in S$ with $x.key = k$, or <code>NIL</code>
<code>INSERT(S, x)</code>	modifying	add the element pointed to by x
<code>DELETE(S, x)</code>	modifying	remove x — takes a pointer, not a key
<code>MINIMUM(S)</code> / <code>MAXIMUM(S)</code>	query	extreme element (requires total order)
<code>SUCCESSOR(S, x)</code> / <code>PREDECESSOR(S, x)</code>	query	next larger / smaller element

Note carefully that `DELETE` takes a pointer to the element, not a key. This convention is what makes the tables below say $O(1)$ in places that look surprising — and it is exactly why deletion from a *singly* linked list is still $\Theta(n)$.

The enumeration convention: one `MINIMUM` followed by `n-1` `SUCCESSOR` calls should enumerate the set in sorted order.

The named ADTs

ADT	OPERATIONS	WHERE
Dictionary	<code>INSERT</code> , <code>DELETE</code> , <code>SEARCH</code>	hash tables M07 , BSTs M08
Container	store/retrieve by <i>insertion order only</i>	stacks, queues (§4)
Priority queue	<code>INSERT</code> , <code>MINIMUM/MAXIMUM</code> , <code>EXTRACT-MIN/MAX</code>	heaps M05

Skiena's distinction is the crisp one [§3.2, p.75]:

*I use the term **container** to denote an abstract data type that permits storage and retrieval of data items **independent of content**. By contrast, **dictionaries** are abstract data types that retrieve **based on key values or content**.*

The best way to implement a dynamic set depends upon the operations that you need to support.

Why not just always use an array?

The advantage is that the algorithms are simple. The downside is that many of these operations have a worst-case running time of $\Theta(n)$. [CLRS p.251]

That trade-off is what the rest of Part III exists to fix.

2. Contiguous vs. Linked — the fundamental distinction

[Skiena §3.1, p.69]

*Data structures can be neatly classified as either **contiguous** or **linked**, depending upon whether they are based on arrays or pointers. **Contiguously allocated** structures are composed of single slabs of memory, and include arrays, matrices, heaps, and hash tables. **Linked** data structures are composed of distinct chunks of memory bound together by pointers, and include lists, trees, and graph adjacency lists.*

The street analogy: an array is a street of equal-sized houses numbered 1 to n ; you compute a house's exact position from its number alone. (*Skiena's footnote: houses in Japanese cities are traditionally numbered in the order they were built, not by physical location — "this makes it extremely difficult to locate a Japanese address without a detailed map." That is a linked structure.*)

The trade-off table

ARRAYS WIN ON	LINKED STRUCTURES WIN ON
Constant-time access given the index — the index maps directly to an address	No overflow unless memory is actually full
Space efficiency — pure data, no links, no end-of-record markers (records are fixed size)	Simpler insertion and deletion
Memory locality — "physical continuity between successive data accesses helps exploit the high-speed cache memory on modern computer architectures"	With large records, moving pointers is easier and faster than moving the items themselves

Take-Home Lesson: Dynamic memory allocation provides us with flexibility on how and where we use our limited storage resources.

Both are recursive objects

- **Lists** – Chopping the first element off a linked list leaves a smaller linked list. This same argument works for **strings**.
- **Arrays** – Splitting the first k elements off an n -element array gives two smaller arrays.

*This insight leads to simpler list processing, and efficient divide-and-conquer algorithms such as **quicksort and binary search**.*

(Same point as the recursive-objects table in [MO1](#) §5.)

3. Arrays, dynamic arrays, and matrices

Address arithmetic [CLRS §10.1.1]

An array starting at address a with elements of b bytes and first index s : element i occupies bytes $a + b(i - s)$ through $a + b(i - s + 1) - 1$.

- $s = 1$: element i at $a + b(i - 1)$.
- $s = 0$: element i at $a + bi$.

Under the RAM model this is $O(1)$ regardless of index.

When elements have different sizes, b is not constant and the formula fails. The usual fix: store **pointers** in the array (pointers are uniformly sized), then follow the pointer.

This costs one extra indirection and destroys locality — the reason

`std::vector<std::string>` traverses worse than `std::vector<int>`.

Dynamic arrays — your first amortized argument

[Skiena §3.1.1, p.70]

Start with size 1; whenever full, allocate $2m$, copy, free the old.

The apparent waste involves recopying the old contents on each expansion. How much work do we really do?

$$\begin{aligned} M &= n + \sum_{i=1}^{\lg n} 2^{i-1} = 1 + 2 + 4 + \dots + n/2 + n \\ &= \sum_{i=0}^{\lg n} n/2^i \leq n \sum_{i=0}^{\infty} 1/2^i = 2n \end{aligned}$$

Thus, each of the n elements moves only two times on average, and the total work of managing the dynamic array is the same $O(n)$ as it would have been if a single array of sufficient size had been allocated in advance!

What you give up:

*The primary thing lost is **the guarantee that each insertion takes constant time in the worst case**. All accesses and most insertions will be fast, except for those relatively few insertions that trigger array doubling. What we get instead is a **promise that the n -th element insertion will be completed quickly enough that the total effort expended so far will still be $O(n)$** . Such **amortized guarantees** arise frequently in the analysis of data structures.*

This is the geometric-series free lunch from M02 §7, and it is formalized in M09.

Engineering note (outside): the growth factor matters. Doubling means freed blocks can never be reused for the next allocation ($1 + 2 + 4 + \dots + 2^{k-1} < 2^k$); growth factors below the golden ratio $\phi \approx 1.618$ allow reuse. `libstdc++` `std::vector` doubles; MSVC uses 1.5×. Also: `reserve()` when you know the size, and note that `push_back` invalidates all iterators on reallocation.

Matrices [CLRS §10.1.2]

For an $m \times n$ matrix M , with s -origin indexing:

LAYOUT	INDEX OF $M[i][j]$ IN A SINGLE ARRAY
Row-major	$s + n(i - s) + (j - s)$; with $s=1$: $n(i-1) + j$; with $s=0$: $ni + j$
Column-major	$s + m(j - s) + (i - s)$; with $s=1$: $i + m(j-1)$; with $s=0$: $i + mj$

Four storage strategies [Fig. 10.1]: single array row-major; single array column-major; array-of-row-arrays ($A[i][j]$); array-of-column-arrays ($A[j][i]$).

Single-array representations are typically more efficient on modern machines than multiple-array representations. But multiple-array representations can sometimes be more flexible, for example allowing for "ragged arrays" in which rows have different lengths.

Block representation. A 4×4 matrix in 2×2 blocks stored as $\langle 1, 2, 5, 6, 3, 4, 7, 8, 9, 10, 13, 14, 11, 12, 15, 16 \rangle$. This is exactly the layout that makes **blocked matrix multiplication** cache-efficient — the $\Theta(n^3)$ algorithm's constant factor can improve several-fold from layout alone.

Engineering note: C/C++ are row-major, Fortran/MATLAB/Julia column-major. Traversing a C matrix column-by-column is catastrophically slower than row-by-row for large n — a pure cache effect the RAM model cannot see.

4. Stacks and Queues

Definitions [CLRS §10.1.3]

Stacks and queues are dynamic sets in which the element removed by `DELETE` is prespecified.

- **Stack:** last-in, first-out (**LIFO**). `INSERT = PUSH`, `DELETE = POP` (takes no argument).
- **Queue:** first-in, first-out (**FIFO**). `INSERT = ENQUEUE`, `DELETE = DEQUEUE`.

Skiena's characterization of when to use which [§3.2, p.75]:

***Stacks** are simple to implement and very efficient. For this reason, stacks are probably the right container to use when retrieval order doesn't matter at all, such as when processing batch jobs. ... Algorithmically, LIFO tends to happen in the course of executing recursive algorithms.*

***Queues** support retrieval in FIFO order. This is surely the fairest way to control waiting times. Jobs processed in FIFO order minimize the maximum time spent waiting. Note that the average waiting time will be the same regardless of whether FIFO or LIFO is used. ... We will see queues later as the fundamental data structure controlling breadth-first search (BFS) in graphs.*

That average-vs-maximum observation is the non-obvious one: FIFO doesn't reduce total or average wait, only the worst wait.

Array implementations

Stack — `S[1..n]` plus `S.top` (index of the most recent element) and `S.size`:

```
STACK-EMPTY(S): return S.top == 0
PUSH(S, x):      if S.top == S.size: error "overflow"
                  S.top = S.top + 1; S[S.top] = x
POP(S):          if STACK-EMPTY(S): error "underflow"
                  S.top = S.top - 1; return S[S.top + 1]
```

Queue — a circular buffer `Q[1..n]` holding at most `n - 1` elements, with `Q.head` (the head) and `Q.tail` (the next free slot):

```
ENQUEUE(Q, x):  Q[Q.tail] = x
                 Q.tail = (Q.tail == Q.size) ? 1 : Q.tail + 1
DEQUEUE(Q):     x = Q[Q.head]
                 Q.head = (Q.head == Q.size) ? 1 : Q.head + 1
                 return x
```

The capacity- $n-1$ subtlety — this is the classic circular-buffer bug. `Q.head == Q.tail` means **empty**. If you allowed `n` elements, a completely full buffer would *also* have `Q.head == Q.tail` and you couldn't distinguish the two. CLRS's fix is to waste one slot. The alternatives are keeping an explicit `count`, or a "last operation was an insert" flag.

Each operation is $O(1)$.

C++ Implementation — ring-buffer deque

```
#include <vector>
#include <stdexcept>
#include <cstdint>

// Fixed-capacity double-ended queue on a ring buffer.
// All operations  $O(1)$ . Holds at most capacity elements (uses an
// explicit
// count, so no slot is wasted and full/empty are unambiguous).
template <typename T>
class RingDeque {
public:
    explicit RingDeque(size_t capacity)
        : buf_(capacity), head_(0), count_(0) {}

    bool empty() const { return count_ == 0; }
    bool full() const { return count_ == buf_.size(); }
    size_t size() const { return count_; }

    void pushBack(const T& x) {
        if (full()) throw overflow_error("deque overflow");
        buf_[ (head_ + count_) % buf_.size() ] = x;
        ++count_;
    }

    void pushFront(const T& x) {
        if (full()) throw overflow_error("deque overflow");
        head_ = (head_ + buf_.size() - 1) % buf_.size(); // +size
        // avoids negative
        buf_[head_] = x;
        ++count_;
    }

    T popFront() {
        if (empty()) throw underflow_error("deque underflow");
```



```

        T x = buf_[head_];
        head_ = (head_ + 1) % buf_.size();
        --count_;
        return x;
    }

    T popBack() {
        if (empty()) throw underflow_error("deque underflow");
        --count_;
        return buf_[(head_ + count_) % buf_.size()];
    }

    const T& front() const {
        if (empty()) throw underflow_error("empty deque");
        return buf_[head_];
    }

    const T& back() const {
        if (empty()) throw underflow_error("empty deque");
        return buf_[(head_ + count_ - 1) % buf_.size()];
    }

private:
    vector<T> buf_;
    size_t head_;    // index of the front element
    size_t count_;   // number of elements currently stored
};

```

Implementation notes.

- `(head_ + buf_.size() - 1) % buf_.size()` rather than `head_ - 1`: with unsigned `size_t`, `0 - 1` wraps to a huge value. This is a real and common bug.
- The **explicit** `count_` removes the wasted slot and the full/empty ambiguity — cleaner than CLRS's `n-1` convention.
- If the capacity is a power of two, replace `% size` with `& (size - 1)`; measurably faster in hot loops.

Queue from two stacks [CLRS Ex. 10.1-7]

```

#include <stack>
#include <stdexcept>

// FIFO queue built from two LIFO stacks.

```

```

// enqueue: O(1) worst case.  dequeue: O(1) AMORTIZED (O(n) worst
// case).
template <typename T>
class QueueFromStacks {
public:
    void enqueue(const T& x) { in_.push(x); }

    T dequeue() {
        if (out_.empty()) {
            if (in_.empty()) throw underflow_error("empty queue");
            // Move everything across once; each element is moved
            // at most twice
            // over its whole lifetime (once in, once out) -> O(1)
            // amortized.
            while (!in_.empty()) { out_.push(in_.top()); in_.pop(); }
        }
        T x = out_.top();
        out_.pop();
        return x;
    }

    bool empty() const { return in_.empty() && out_.empty(); }
    size_t size() const { return in_.size() + out_.size(); }

private:
    stack<T> in_, out_;
};

```

The analysis is the point. Each element is pushed to `in_` once, moved to `out_` at most once, popped from `out_` once — **3 stack operations per element over its entire lifetime**, so $O(1)$ amortized despite $O(n)$ worst case for a single `dequeue`. This is the **aggregate method** of M09, and a very common interview question.

(The reverse — a stack from two queues [Ex. 10.1-8] — is genuinely worse: one of push/pop must be $\Theta(n)$, with no amortized rescue.)

Two stacks in one array [CLRS Ex. 10.1-3]

Grow one from index `1` upward and the other from index `n` downward. Neither overflows until their **combined** size reaches `n`. `PUSH/POP` stay $O(1)$. A nice trick for memory-constrained settings, and the idea behind arena allocators that allocate from both ends.

5. Linked Lists

Anatomy [CLRS §10.2]

Each element of a **doubly linked list** has `key`, `next`, `prev`. `x.prev = NIL` $\Rightarrow$ `x` is the **head**; `x.next = NIL` $\Rightarrow$ `x` is the **tail**. `L.head` points at the first element; `L.head = NIL` $\Rightarrow$ empty.

Four independent axes: singly vs doubly linked · sorted vs unsorted · circular vs not · with vs without a sentinel.

*Since the elements often contain keys that can be searched for, linked lists are sometimes called **search lists**.*

The core operations

```
LIST-SEARCH(L, k)                //  $\Theta(n)$ 
1  x = L.head
2  while x  $\neq$  NIL and x.key  $\neq$  k
3      x = x.next
4  return x

LIST-PREPEND(L, x)                //  $O(1)$ 
1  x.next = L.head; x.prev = NIL
3  if L.head  $\neq$  NIL: L.head.prev = x
5  L.head = x

LIST-INSERT(x, y)                 //  $O(1)$ : splice x in immediately after y
1  x.next = y.next; x.prev = y
3  if y.next  $\neq$  NIL: y.next.prev = x
5  y.next = x

LIST-DELETE(L, x)                 //  $O(1)$  given the pointer
1  if x.prev  $\neq$  NIL: x.prev.next = x.next
3  else: L.head = x.next
```

```
4  if x.next ≠ NIL:  x.next.prev = x.prev
```

→ **C++ implementation:** [A1 LIST-SEARCH](#), [LIST-PREPEND](#), [LIST-INSERT](#), [LIST-DELETE](#)

Array vs list, stated precisely [CLRS p.261]

*Insertion and deletion are **faster** on doubly linked lists than on arrays. If you want to insert a new first element into an array or delete the first element, maintaining relative order, then each existing element must move by one position — $\Theta(n)$ versus $O(1)$.*

*If, however, you want to find the **k -th element** in the linear order, it takes just $O(1)$ in an array regardless of k , but in a linked list you'd have to traverse k elements, taking $\Theta(k)$.*

Sentinels

A **sentinel** `L.nil` is a dummy object with all the attributes of a real node, replacing every `NIL` reference. The list becomes **circular and doubly linked**, with `L.nil` sitting between head and tail. `L.head` is eliminated — the head is `L.nil.next`, the tail is `L.nil.prev`. An empty list is just the sentinel pointing at itself.

Deletion collapses to two lines:

```
LIST-DELETE'(x)
1  x.prev.next = x.next
2  x.next.prev = x.prev

LIST-INSERT'(x, y)          // insert x after y
1  x.next = y.next;  x.prev = y
3  y.next.prev = x;   y.next = x
```

→ **C++ implementation:** [A2 LIST-SEARCH'](#), [LIST-DELETE'](#), [LIST-INSERT'](#) (the sentinel versions)

*No separate procedure for prepending is necessary: to insert at the **head**, let y be `L.nil`; to insert at the **tail**, let y be `L.nil.prev`.*

The sentinel search trick. `LIST-SEARCH` makes **two** comparisons per iteration: "have we run off the end?" and "is this the key?". Store the key **in the sentinel** first and the first test becomes unnecessary — the search is guaranteed to terminate:

```

LIST-SEARCH'(L, k)
1  L.nil.key = k                // guarantee the key is "in"
the list
2  x = L.nil.next
3  while x.key ≠ k:  x = x.next
5  if x == L.nil:  return NIL    // only found it in the
sentinel
7  else:           return x

```

→ **C++ implementation:** [A2 LIST-SEARCH', LIST-DELETE', LIST-INSERT'](#) (the sentinel versions)

CLRS's honest verdict, worth quoting because it resists over-engineering:

*Sentinels often simplify code and might speed up code by a small constant factor, but they **don't typically improve the asymptotic running time**. Use them judiciously. When there are many small lists, the extra storage used by their sentinels can represent **significant wasted memory**. In this book, we use sentinels only when they significantly simplify the code.*

C++ Implementation — doubly linked list with sentinel

```

#include <cstddef>
#include <utility>

// Circular doubly linked list with a sentinel node.
// insert/erase are O(1) given an iterator; search is O(n).
template <typename T>
class SentinelList {
    struct Node {
        T value;
        Node* prev;
        Node* next;
        Node() : value(), prev(this), next(this) {}           // the
sentinel
        Node(const T& v, Node* p, Node* n) : value(v), prev(p),
next(n) {}
    };

public:
    SentinelList() : nil_(new Node()), size_(0) {}

```

```

~SentinelList() { clear(); delete nil_; }

SentinelList(const SentinelList&) = delete;           //
keep it simple
SentinelList& operator=(const SentinelList&) = delete;

using iterator = Node*;

iterator begin() const { return nil_>next; }
iterator end()    const { return nil_; }              // the
sentinel IS end()
bool empty()      const { return nil_>next == nil_; }
size_t size()    const { return size_; }

// Insert before position pos. Returns an iterator to the new
node.
iterator insert(iterator pos, const T& v) {
    Node* n = new Node(v, pos->prev, pos);
    pos->prev->next = n;                               // no
NIL checks needed
    pos->prev = n;
    ++size_;
    return n;
}
void pushFront(const T& v) { insert(begin(), v); }
void pushBack(const T& v)  { insert(end(), v); }

// Erase the node at pos. Returns an iterator to the following
node.
iterator erase(iterator pos) {
    Node* nxt = pos->next;
    pos->prev->next = pos->next;                         //
LIST-DELETE'
    pos->next->prev = pos->prev;
    delete pos;
    --size_;
    return nxt;
}

// Linear search. Returns end() if absent.
iterator find(const T& v) const {

```

```

        nil_->value = v; //
sentinel trick:
        iterator node = nil_->next; //
one comparison per step
        while (!(node->value == v)) node = node->next;
        return node; //
== end() if not found
    }

void clear() { while (!empty()) erase(begin()); }

static const T& valueOf(iterator it) { return it->value; }

private:
    Node* nil_;
    size_t size_;
};

```

Implementation notes.

- The sentinel's constructor sets `prev = next = this` — an empty circular list.
- **No `NIL` checks anywhere** in `insert` or `erase`. That is the whole point.
- `end()` is the sentinel, matching STL convention exactly (this is essentially how `std::list` works).
- `find` writes into the sentinel's `value`, so `nil_` is `mutable` in spirit — hence `value` is written even in a `const` method via the non-const `Node*`. In production you would drop the trick and just check `x != nil_`.

Common bugs

- Forgetting to update `L.head` when deleting the first element (non-sentinel version).
- Updating `next` before reading `prev` (or vice versa) — save what you need first.
- Deleting the sentinel. *"You should never delete the sentinel `L.nil` unless you are deleting the entire list!"*
- Assuming `DELETE` is `O(1)` on a **singly** linked list.

Deleting from a singly linked list in $O(1)$ — and what it breaks

[Skiena §3.3 footnote 3, Fig. 3.2]

The problem: `DELETE(x)` needs `x`'s **predecessor**, which costs $\Theta(n)$ to find in a singly linked list.

The trick: don't delete `x` — **overwrite `x` with the contents of `x.next`**, then delete the node `x.next` originally pointed to.

```
// Delete node x from a singly linked list in O(1), without its
predecessor.
// PRECONDITION: x is not the last node. (Use a permanent sentinel
tail node
// so this always holds.)
template <typename Node>
void deleteWithoutPredecessor(Node* x) {
    Node* victim = x->next;
    x->value = victim->value;           // x now impersonates its
successor
    x->next = victim->next;
    delete victim;                     // the successor's node is
freed instead
}
```

*Special care must be taken if `x` is the first node in the list, or the last node (by employing a permanent sentinel element that is always the last node). **But this would prevent us from having constant-time minimum/maximum operations, because we no longer have time to find new extreme elements after deletion.***

The lesson: a clever trick for one operation can silently break another. Note also that any external pointer to `x.next` is now dangling — a real hazard.

XOR linked lists [CLRS Ex. 10.2-6]

Store one pointer field `x.np = x.next XOR x.prev` instead of two. Traversal works because if you know the previous node's address `p`, then `next = x.np XOR p`. **Reversal is $O(1)$** — just swap the two ends you started from.

Do not use this. It defeats garbage collectors, debuggers, and address sanitizers, and is undefined behaviour in standard C++ (you cannot round-trip a pointer through an integer XOR portably). It is a good puzzle and a bad structure.

6. The dictionary comparison tables

These two tables are the heart of the module. Learn to **derive** them, not memorize them.

Arrays [Skiena §3.3, p.77]

DICTIONARY OPERATION	UNSORTED ARRAY	SORTED ARRAY
<code>Search(A, k)</code>	$O(n)$	$O(\log n)$
<code>Insert(A, x)</code>	$O(1)$	$O(n)$
<code>Delete(A, x)</code>	$O(1)^*$	$O(n)$
<code>Successor(A, x)</code>	$O(n)$	$O(1)$
<code>Predecessor(A, x)</code>	$O(n)$	$O(1)$
<code>Minimum(A)</code>	$O(n)$	$O(1)$
<code>Maximum(A)</code>	$O(n)$	$O(1)$

The starred entry. Deleting `A[x]` leaves a hole. Shifting `A[x+1..n]` down is $\Theta(n)$.

Instead: overwrite `A[x]` with `A[n]` and decrement `n`. $O(1)$. (This destroys order, which is fine in an unsorted array — and is exactly the `pop_back`-swap idiom used in game engines and ECS systems.)

Why `successor` is $O(n)$ in an unsorted array — the entry people get wrong:

The answer is **not** simply `A[x-1]` (or `A[x+1]`), because in an unsorted array an element's **physical** predecessor is not necessarily its **logical** predecessor. The predecessor of `A[x]` is the biggest element smaller than `A[x]`.

Why you can't cache the min and max in an unsorted array:

It is tempting to set aside extra variables containing the current minimum and maximum, so we can report them in $O(1)$. **But this is incompatible with constant-time deletion**, as deleting the minimum-valued item mandates a linear-time search to find the new minimum.

*Take-Home Lesson: Data structure design must **balance all the different operations** it supports. The fastest data structure to support both operations *A* and *B* may well not be the fastest structure to support just operation *A* or *B*.*

Linked lists [Skiena §3.3, p.79]

DICTIONARY OPERATION	SINGLY UNSORTED	SINGLY SORTED	DOUBLY UNSORTED	DOUBLY SORTED
Search(L, k)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert(L, x)	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Delete(L, x)	$O(n)^*$	$O(n)^*$	$O(1)$	$O(1)$
Successor(L, x)	$O(n)$	$O(1)$	$O(n)$	$O(1)$
Predecessor(L, x)	$O(n)$	$O(n)^*$	$O(n)$	$O(1)$
Minimum(L)	$O(1)^*$	$O(1)$	$O(n)$	$O(1)$
Maximum(L)	$O(1)^*$	$O(1)^*$	$O(n)$	$O(1)$

The four things this table teaches:

1. The predecessor-pointer problem drives everything.

*The definition of `Delete` states we are given a pointer `x` to the item to be deleted. But what we really need is a pointer to the element **pointing to** `x`, because that is the node that needs to be changed. **We can do nothing without this list predecessor, and so must spend linear time searching for it on a singly linked list.***

2. Sorting helps lists far less than it helps arrays.

***Binary search is no longer possible**, because we can't access the median element without traversing all the elements before it. What sorted lists do provide is **quick termination of unsuccessful searches** — if we have not found Abbott by the time we hit Costello, he doesn't exist in the list. Still, searching takes linear time in the worst case.*

3. Deletion from a sorted doubly linked list beats a sorted array. Splicing out a node is $O(1)$; filling the hole in an array by shifting is $\Theta(n)$.

4. The `Minimum/Maximum` starred entries are a genuine accounting insight. How can `Maximum` be $O(1)$ on an unsorted singly linked list?

We can maintain a **separate pointer** to the list tail, provided we pay the maintenance costs on every insertion and deletion. ... We have no efficient way to find this predecessor for singly linked lists. So why can we implement `Maximum` in `O(1)`? **The trick is to charge the cost to each deletion, which already took linear time.** Adding an extra linear sweep to update the pointer **does not harm the asymptotic complexity of `Delete`**, while gaining us `Maximum` in constant time as a reward for clear thinking.

That is amortized/charging reasoning in miniature: a free operation because a *different* operation was already paying $\Theta(n)$.

7. Basic priority-queue implementations

[Skiena §3.5, p.87] — the array-and-tree versions, complementing the heap in M05 §3.

Why a PQ rather than sorting

The priority queue provides more flexibility than simple sorting, because it allows **new elements to enter a system at arbitrary intervals**. It can be much more cost-effective to insert a new job into a priority queue than to re-sort everything on each such arrival.

(Skiena's memorable model: a "little black book" priority queue keyed on desirability — `Find-Maximum`, spend an evening evaluating, then reinsert with a revised score.)

The table

OPERATION	UNSORTED ARRAY	SORTED ARRAY	BALANCED TREE
<code>Insert(Q, x)</code>	<code>O(1)</code>	<code>O(n)</code>	<code>O(log n)</code>
<code>Find-Minimum(Q)</code>	<code>O(1)</code>	<code>O(1)</code>	<code>O(1)</code>
<code>Delete-Minimum(Q)</code>	<code>O(n)</code>	<code>O(1)</code>	<code>O(log n)</code>

Two tricks are hiding in that table.

`Delete-Minimum` in `O(1)` on a sorted array. Store the array in **reverse** order, largest on top. The minimum is then the *last* element, and deleting it moves nothing — just decrement `n`.

Find-Minimum in $O(1)$ everywhere. Keep an extra variable pointing at the minimum.

*Updating this pointer on each insertion is easy — we update it iff the newly inserted value is less than the current minimum. But what happens on a `delete-minimum`? We can delete that minimum element we point to, and then do a search to restore this cached value. **The operation to identify the new minimum takes linear time on an unsorted array and logarithmic time on a tree, and hence can be folded into the cost of each deletion.***

Same charging argument as **Maximum** on the singly linked list. **When an operation is already paying $\Theta(f(n))$, you can bolt on anything cheaper for free.**

Take-Home Lesson:** Building algorithms around data structures such as dictionaries and priority queues leads to **both clean structure and good performance.

8. Representing rooted trees

[CLRS §10.3]

Binary trees

Each node `x` has `x.p` (parent), `x.left`, `x.right`. `x.p = NIL` $\Rightarrow$ root. `T.root` points at the root; `T.root = NIL` $\Rightarrow$ empty.

Bounded branching

For at most `k` children per node, replace `left/right` with `child1, ..., childk`. But:

*This scheme no longer works when the number of children is **unbounded**, since we do not know how many attributes to allocate in advance. Moreover, if `k` is bounded by a large constant but most nodes have few children, **we may waste a lot of memory.***

Left-child, right-sibling — $O(n)$ space for any branching

Two pointers per node (plus a parent pointer):

1. `x.left-child` $\rightarrow$ the **leftmost child** of `x`
2. `x.right-sibling` $\rightarrow$ the sibling **immediately to `x`'s right**

`x.left-child = NIL` $\Rightarrow$ no children; `x.right-sibling = NIL` $\Rightarrow$ `x` is the rightmost child.

*It has the advantage of using **only $O(n)$ space** for any n -node rooted tree.*

How to iterate a node's children: follow `left-child`, then chase `right-sibling` repeatedly. Linear in the number of children, exactly as you'd want.

```
#include <vector>
#include <functional>

// Left-child / right-sibling representation of an arbitrary rooted
// tree.
// O(n) space regardless of branching factor.
template <typename T>
struct LCRSNode {
    T value;
    LCRSNode* parent      = nullptr;
    LCRSNode* leftChild   = nullptr;
    LCRSNode* rightSibling= nullptr;
    explicit LCRSNode(const T& v) : value(v) {}
};

// Add c as a child of p. O(1) -- new children go at the FRONT of
// the list.
template <typename T>
void addChild(LCRSNode<T>* p, LCRSNode<T>* c) {
    c->parent = p;
    c->rightSibling = p->leftChild;
    p->leftChild = c;
}

// Preorder traversal, O(n) total (CLRS Ex. 10.3-4).
template <typename T, typename Visit>
void traverse(LCRSNode<T>* x, Visit visit) {
    for (; x != nullptr; x = x->rightSibling) {
        visit(x->value);
        traverse(x->leftChild, visit);
    }
}
```

Other representations

*In Chapter 6 we represented a **heap** by a single array plus the index of the last node. The trees in Chapter 19 [**union-find**] are traversed **only toward the root**, and so only the parent pointers are present: there are no pointers to children. Many other schemes are possible. **Which scheme is best depends on the application.***

REPRESENTATION	SPACE	GOOD FOR
left/right/p	3n pointers	binary trees, BSTs
child ₁ ...child _k	kn pointers	bounded, dense branching
left-child, right-sibling	3n pointers	unbounded branching
implicit array (heap)	0 pointers	complete binary trees only
parent only	n pointers	union-find (M10)
adjacency list	O(n + m)	general graphs (M13)

Traversal without recursion or extra space

[CLRS Ex. 10.3-5] asks for an $O(n)$ traversal using $O(1)$ **extra space** without modifying the tree. This is possible only if nodes have parent pointers — you track where you came from. (With no parent pointers, the **Morris traversal** achieves it by temporarily rewiring threads, but that modifies the tree during the procedure. See M08.)

9. Specialized data structures — the map ahead

[Skiena §3.8, p.98]

*The basic data structures described thus far all represent an **unstructured set** of items so as to facilitate retrieval operations. Not as well known are data structures for representing **more specialized kinds of objects**, such as points in space, strings, and graphs.*

The design principles are the same: there exists a set of basic operations we need to perform repeatedly; we seek a data structure that allows these operations to be performed very efficiently.

OBJECT	STRUCTURES	MODULE
Strings	character arrays; suffix trees / suffix arrays for fast pattern matching	M18 <i>(planned)</i>
Geometric	polygons as vertex arrays $(v_1, \dots, v_n, v_1)$; kd-trees organizing points by location	M26 <i>(planned)</i>
Graphs	adjacency matrices or adjacency lists — "the choice of representation can have a substantial impact on the design of the resulting graph algorithms"	M13
Sets	dictionaries for membership; bit vectors where bit i is 1 iff i is in the subset	M07

10. War Story: Stripping Triangulations

[Skiena §3.6, p.89] — *the illustration of "choosing the right data structure is the key to getting the time complexity down".*

The problem. 3-D models are triangulated surfaces. Rendering hardware is so fast that **the bottleneck is feeding the triangulation into the hardware**. Instead of sending three vertices per triangle, send **triangle strips**: adjacent triangles share two vertices, so each additional triangle costs only one new vertex. Task: cover all triangles with as few strips as possible, without overlap.

Modeling it. Build the **dual graph** — one vertex per triangle, an edge between adjacent triangles. Partitioning the dual graph's vertices into as few paths as possible. Partitioning into **one** path would be a **Hamiltonian path**, which is NP-complete —

...so we knew not to look for an optimal algorithm, but concentrate instead on heuristics.

(That move — recognize the NP-complete core, then stop looking for optimality — is the whole content of [M19 \(planned\)](#)/[M20 \(planned\)](#) applied in one sentence.)

The naive heuristic. Start anywhere, walk left–right until you hit the boundary or a used triangle. Fast, simple, no quality guarantee.

The greedy heuristic. Repeatedly peel off the **longest** remaining strip.

Being greedy does not guarantee the best solution overall, since the first strip you peel off might break apart a lot of potential strips we would have wanted to use later. Still, being greedy is a good rule of thumb if you want to get rich.

The performance problem. Naively: $O(kn)$ to walk from all n vertices, repeated for $\approx n/k$ strips = $O(n^2)$ — "hopelessly slow on even a small model of 20,000 triangles."

The fix — two data structures working together:

STRUCTURE	WHY
Priority queue on strip lengths	The next strip to peel is always at the top. Must support reducing the priority of arbitrary elements when peeling shortens other strips. Since strip lengths were bounded by 256 (a hardware constraint), they used a bounded-height priority queue — an array of buckets. <i>"An ordinary heap would also have worked just fine."</i>
Dictionary from triangle → queue position	To update a triangle's queue entry you must find it. Integrating the dictionary with the priority queue gives a structure supporting the whole operation set.

That is exactly the handle problem from M05 §4 — DECREASE-KEY needs an object→position map.

Run time improved by several orders of magnitude after employing this data structure.

The measured results [Fig. 3.9]:

MODEL	TRIANGLES	NAIVE COST	GREEDY COST	GREEDY TIME
Diver	3,798	8,460	4,650	6.4 s
Heads	4,157	10,588	4,749	9.9 s
Framework	5,602	9,274	7,210	9.7 s
Bart Simpson	9,654	24,934	11,676	20.5 s
Enterprise	12,710	29,016	13,738	26.2 s
Torus	20,000	40,000	20,200	272.7 s
Jaw	75,842	104,203	95,020	136.2 s

Savings of 10–50%, "quite remarkable since the greatest possible improvement (going from three vertices per triangle down to one) yields a savings of only 66.6%."

And note the **Torus vs Jaw anomaly**: the final algorithm is $O(n \cdot k)$ where k is the average strip length. The torus has 20,000 triangles but very long strips (272.7 s); the jaw has 75,842 triangles but short ones (136.2 s). **The complexity parameter that mattered wasn't n .**

Three lessons:

1. With a large enough dataset, **only linear or near-linear algorithms are likely to be fast enough.**
 2. **Choosing the right data structure is often the key** to getting the time complexity down.
 3. Greedy can significantly improve on naive — but **how much can only be determined by experimentation.**
-

11. War Story: String 'em Up — the data-structure escalation ladder

[Skiena §3.9, p.98] — the best worked example of *"isolate the inner-loop operation and optimize the structure that supports it."*

The problem. Sequencing by hybridization: given all length- k substrings of an unknown DNA string s , construct all consistent length- $2k$ strings. The naive method concatenates all $O(n^2)$ pairs of k -strings and checks that all $k-1$ substrings **straddling the join** are in the dictionary.

So the inner-loop operation is: **is this length- k string in our dictionary?** — performed $k-1$ times for each of $O(n^2)$ concatenations.

The escalation:

ATTEMPT	PER-LOOKUP COST	WHAT HAPPENED
Binary search tree	$O(k \log n)$	<i>"It takes forever on string lengths of only 2,000 characters."</i>
Hash table	$O(k)$	<i>"About ten times faster on strings of length 2,000. So now we can get up to about 4,000. Big deal."</i>
Suffix tree	$O(1)$ per subsequent test	Worked — then ran out of memory at $\Theta(n^2)$ nodes
Compressed suffix tree	$O(1)$, linear space	Reached $n = 65,536$

The insight that unlocked the suffix tree is the important part:

"Sure, it takes k comparisons to test the first substring. But maybe we can do better on the second test. **Remember where our dictionary queries are coming from.** When we concatenate `ABCD` with `EFGH`, we are first testing whether `BCDE` is in the dictionary, then `CDEF`. **These strings differ from each other by only one character.** We should be able to exploit this so each subsequent test takes constant time..."

"We can't do that with a hash table. The second key is not going to be anywhere near the first in the table. A binary search tree won't help either. Since `ABCD` and `BCDE` differ in the first character, the two strings will be in different parts of the tree."

"But we can use a **suffix tree**... By following a pointer from `ACAC` to its longest proper suffix `CAC`, we get to the right place to test whether `CACT` is in our set of strings. **One character comparison is all we need to do from there.**"

That suffix-link idea is the same one behind Aho–Corasick and KMP (M18 (planned)).

The measured escalation [Fig. 3.14, seconds]:

N	BINARY TREE	HASH TABLE	SUFFIX TREE	COMPRESSED
256	17.1	9.4	3.8	0.2
512	31.6	67.0	6.9	1.3
1,024	1,828.9	96.6	31.5	2.7
2,048	11,441.7	941.7	553.6	39.0
4,096	> 2 days	5,246.7	out of memory	45.4
8,192	> 2 days	—	—	642.0
65,536	—	—	—	39,776.9

We isolated a single operation that was being performed repeatedly and optimized the data structure to support it. When an improved dictionary structure still did not suffice, we looked deeper into the kind of queries we were performing, so that we could identify an even better data structure. Finally, we didn't give up until we had achieved the level of performance we needed. In algorithms, as in life, persistence usually pays off.

Why this matters for engineering: the profiler told them *where* the time went; only understanding the **query pattern** told them what structure to use. A profiler cannot tell you that consecutive queries differ by one character.

Chapter in One Page

CONCEPT	THE ONE-LINE VERSION
Dynamic set	A set that grows/shrinks; operations: SEARCH, INSERT, DELETE, MIN/MAX, SUCC/PRED.
DELETE takes a pointer	Not a key. This is why singly-linked deletion is still $\Theta(n)$.
Container vs dictionary	Container retrieves by insertion order ; dictionary by content .
Contiguous vs linked	Arrays: $O(1)$ index, no overhead, cache-friendly, fixed size. Lists: $O(1)$ splice, unbounded, pointer overhead, scattered.
Both are recursive	Chop the head off a list; split an array in two.
Dynamic array	Doubling gives $\leq 2n$ total moves — $O(1)$ amortized , not worst case.
Row- vs column-major	C is row-major; traversing the wrong way destroys cache performance.
Block layout	Store $k \times k$ blocks contiguously — makes blocked matrix multiply cache-efficient.
Stack	LIFO; the right container when order doesn't matter; arises from recursion.
Queue	FIFO; minimizes maximum wait, not average ; drives BFS.
Circular buffer	<code>head == tail</code> means empty; either waste one slot or keep an explicit count.
Queue from 2 stacks	$O(1)$ amortized — each element moves at most 3 times total.
Two stacks in one array	Grow from both ends; neither overflows until the total reaches n .
Doubly linked list	INSERT/DELETE $O(1)$ given a pointer; k -th element is $\Theta(k)$.
Sentinel	Circular list with a dummy node; kills every <code>NIL</code> check. Judiciously — many small lists waste memory.
Sentinel search trick	Put the key in the sentinel; one comparison per iteration instead of two.
Singly-linked delete	Copy the successor's contents over <code>x</code> , delete the successor. Breaks <code>Min/Max</code> and dangles pointers.
Unsorted array delete is $O(1)$	Overwrite <code>A[x]</code> with <code>A[n]</code> , decrement <code>n</code> .
Successor in an	$\Theta(n)$ — physical neighbour $\neq$ logical neighbour.

unsorted array

Can't cache min/max in an unsorted array Incompatible with $O(1)$ deletion.

The predecessor-pointer problem Singly-linked **DELETE** needs the predecessor $\Rightarrow \Theta(n)$. This is why doubly linked lists exist.

Sorting helps lists less No binary search without random access; you only gain early termination.

The charging trick **Maximum** is free on a list whose **Delete** was already $\Theta(n)$.

PQ **Find-Min** in $O(1)$ Cache a pointer; restoring it folds into the deletion cost.

PQ **Delete-Min** on a sorted array Store **reversed**; the min is last; deleting it moves nothing.

Left-child right-sibling $O(n)$ space for **any** branching factor; 2 pointers + parent.

Tree representations pointers / bounded children / LCRS / implicit array / parent-only / adjacency list.

Specialized structures suffix trees (strings), kd-trees (geometry), adjacency lists (graphs), bit vectors (sets).

Triangle strips lesson PQ + dictionary together turned $O(n^2)$ into $O(nk)$ — orders of magnitude.

String 'em up lesson BST $\rightarrow$ hash $\rightarrow$ suffix tree $\rightarrow$ compressed suffix tree.
Understand the query pattern, not just the profile.

The master trade-off The best structure for A **and** B is usually not the best for A or B alone.

Recognition Table

CLUE	STRUCTURE
Fixed size, index-based access, iteration-heavy	array / <code>std::vector</code>
Frequent insert/delete in the middle, have the position	doubly linked list / <code>std::list</code>
Frequent insert/delete at both ends	deque / ring buffer / <code>std::deque</code>
LIFO / matching brackets / undo / DFS / expression evaluation	stack
FIFO / BFS / task queue / producer-consumer	queue
Sliding-window maximum	monotonic deque
"Next greater element"	monotonic stack
Need min and max and insert, all fast	can't do it with one array — heap or balanced tree
Repeatedly extract the extremum with changing priorities	priority queue + an object → index dictionary
Priorities are small bounded integers	bucket queue (array of buckets), not a heap
Unbounded branching factor in a tree	left-child, right-sibling
Only need to walk toward the root	parent pointers only (union-find)
Complete binary tree	implicit array — no pointers at all
Queries differ from each other by one character	suffix tree / suffix links , not a hash table
Membership over a dense small integer universe	bit vector
Profiler says one inner-loop lookup dominates	escalate the dictionary structure
Matrix traversed by columns in C	transpose it or change the layout

Common Mistakes Recap

1. Believing dynamic-array `push_back` is $O(1)$ **worst case**. It is $O(1)$ amortized.
2. Circular buffer where full and empty both give `head == tail`.
3. `head - 1` with unsigned arithmetic instead of `(head + size - 1) % size`.
4. Assuming `DELETE` is $O(1)$ on a singly linked list.
5. Forgetting to update `L.head` when deleting the first element without a sentinel.
6. Deleting the sentinel.

7. Using the $O(1)$ singly-linked delete trick and then wondering why `Maximum` broke, or why an external pointer dangles.
 8. Thinking `Successor` in an unsorted array is `A[x+1]`.
 9. Caching min/max alongside $O(1)$ deletion in an unsorted array.
 10. Using `left/right` child pointers for a tree with unbounded branching.
 11. Traversing a C matrix column-by-column for large `n`.
 12. Reaching for a heap when priorities are small bounded integers (a bucket queue is $O(1)$).
 13. Forgetting that `decrease-key` needs a position map — `std::priority_queue` has none.
 14. Optimizing the data structure without first understanding the **query pattern**.
-

Self-Test

1. List the six dynamic-set operations. Why does `DELETE` take a pointer rather than a key? (§1)
2. Give three advantages of arrays and three of linked structures. (§2)
3. Derive the $2n$ bound for dynamic-array doubling. What guarantee does it give up? (§3)
4. Give the row-major and column-major index formulas. When does block layout matter? (§3)
5. In a circular-buffer queue, why does CLRS's version hold only $n - 1$ elements? Give two alternatives. (§4)
6. Implement a queue with two stacks and analyze it. Why is a stack from two queues worse? (§4)
7. What exactly does a sentinel buy you, and what does it cost? (§5)
8. Explain the sentinel search trick. How many comparisons per iteration does it save? (§5)
9. Delete a node from a singly linked list in $O(1)$ without its predecessor. What breaks? (§5)
10. Why is `Delete` $O(1)$ on an **unsorted** array? Why is `Successor` $\Theta(n)$? (§6)
11. Why can't you cache the minimum in an unsorted array with $O(1)$ deletion? (§6)
12. Why is `Delete` $\Theta(n)$ on a singly linked list even given a pointer to the node? (§6)

13. How can `Maximum` be $O(1)$ on an unsorted **singly** linked list? (§6)
 14. How do you get `Find-Minimum` in $O(1)$ for all three PQ implementations? (§7)
 15. How does a sorted array give `Delete-Minimum` in $O(1)$? (§7)
 16. Describe left-child/right-sibling. Why $O(n)$ space for any branching factor? (§8)
 17. Name five ways to represent a rooted tree and when each is best. (§8)
 18. In the triangle-strip story, which two structures were combined and why did each matter? (§10)
 19. Why did the torus take longer than the jaw despite having a quarter as many triangles? (§10)
 20. Trace the four-structure escalation in "String 'em Up". What property of the *queries* made the suffix tree the right answer? (§11)
-

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
Build a linked list from scratch	707 · Design Linked List	<code>LIST-INSERT</code> , <code>LIST-DELETE</code> , <code>LIST-SEARCH</code> with every boundary case the pseudocode hides
Hash map + doubly linked list	146 · LRU Cache	the interview data-structure question: <code>O(1)</code> delete needs the <i>pointer</i> , which is why the map stores iterators
Same, one level harder	460 · LFU Cache	a list of lists; teaches why sentinels stop being optional
Stack with an extra invariant	155 · Min Stack	an auxiliary stack is the whole trick — <code>O(1)</code> min without scanning
One ADT on top of another	225 · Implement Stack using Queues	forces you to state the ADT contract separately from its implementation
Circular buffer	622 · Design Circular Queue	the modular-arithmetic wraparound and the full-vs-empty ambiguity
Amortized array growth	380 · Insert Delete GetRandom O(1)	<code>vector</code> + index map; the swap-with-last deletion trick
Trees as pointers	104 · Maximum Depth of Binary Tree · 226 · Invert Binary Tree	the left-child/right-sibling and pointer-chasing habits of §8

Beyond LeetCode. [CSES Problem Set](#) — *Introductory Problems* and *Range Queries*.
[Codeforces](#) `data structures` tag · `implementation` tag.

The drill that matters here: every time you write a pointer-based structure, run it under `-fsanitize=address,undefined`. The appendix code below was developed that way, and it is the difference between "it passed my tests" and "it is correct".

C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., §1.4–1.5 — the chapter's most important section for this module.

1. Pointers, `new`, `delete`, and the leak

Weiss [§1.5.1, p.22]: "C++ *does not have garbage collection*. When an object that is allocated by `new` is no longer referenced, the `delete` operation must be applied to the object (through a pointer). Otherwise, the memory that it consumes is lost... This is known as a **memory leak**."

And his most useful rule, stated plainly:

"One important rule is to not use `new` when an automatic variable can be used instead."

A linked list is one of the few structures where you genuinely need `new`, because nodes must outlive the function that created them and must not move when the container grows. That is precisely the property a `vector` cannot give you — and it is the real reason `LIST-DELETE` can be $O(1)$ while `vector::erase` cannot.

Use `nullptr`, never `NULL` or `0`: `nullptr` has its own type and cannot be mistaken for an integer in overload resolution.

2. The Big-Five — and why this module cannot accept the defaults

Weiss [§1.5.6, p.30]: every class comes with five compiler-generated operations — **destructor, copy constructor, move constructor, copy assignment, move assignment**. His rule:

"Either you accept the default for all five operations, or you should declare all five, and explicitly define, `default` (use the keyword `default`), or disallow each (use the keyword `delete`)."

And exactly when the defaults break [p.32]:

*"The main problem occurs in a class that contains a data member that is a **pointer**... the copy constructor and copy assignment operator both copy the **value of the pointer** rather than the objects being pointed at. Thus, we will have two class instances that contain pointers that point to the same object. This is a so-called **shallow copy**."*

A list class holding `Node* head_` is precisely that case. Copy it with the defaults and you get two lists sharing nodes, then a **double free** when both destructors run. The appendix's `PlainList` therefore declares all five:

```

class PlainListSketch {
public:
    PlainListSketch() = default;
    ~PlainListSketch(); //
    frees every node
    PlainListSketch(const PlainListSketch&) = delete; //
    no shallow copy
    PlainListSketch& operator=(const PlainListSketch&) = delete;
    PlainListSketch(PlainListSketch&&) noexcept; //
    moving IS safe
    PlainListSketch& operator=(PlainListSketch&&) noexcept;
private:
    struct ListNode { int key; ListNode* next; };
    ListNode* head_ = nullptr;
};

```

Deleting the copy operations is an honest choice, not laziness: a deep copy of a list is $\Theta(n)$ and almost never what the caller meant. Making it a compile error is better than making it silent. Moving stays legal because it is a pointer steal.

3. `noexcept` on move operations

`vector` will only *move* your objects when it reallocates if the move constructor is `noexcept`; otherwise it **copies**, to preserve the strong exception guarantee. For a type whose move is a pointer steal, forgetting `noexcept` silently costs you the entire benefit.

4. `->` versus `.`

Weiss [§1.5.1, p.22]: "If a pointer variable points at a class type, then a (visible) member of the object being pointed at can be accessed via the `->` operator." So `x->next` is `(*x).next`. In this module every traversal is `x = x->next`, and every `NIL` in the pseudocode is `nullptr` in the code.

5. Iterator invalidation — the rule that decides which container you pick

This is the practical heart of "contiguous vs linked":

CONTAINER	INSERT/ERASE IN THE MIDDLE	INVALIDATES
<code>vector</code>	$O(n)$	all iterators/pointers on reallocation; everything after the point on erase
<code>deque</code>	$O(n)$	all iterators; references survive end-insertions
<code>list</code> / <code>forward_list</code>	$O(1)$ given the position	only the erased element
<code>map</code> / <code>set</code>	$O(\lg n)$	only the erased element
<code>unordered_map</code>	$O(1)$ expected	all iterators on rehash; references survive

`std::list::splice` moves elements between lists in $O(1)$ and invalidates nothing — it is `LIST-DELETE` + `LIST-INSERT` with the standard library's blessing, and it is how you write an LRU cache without hand-rolling nodes.

6. What the standard library gives you for free

CLRS STRUCTURE	C++
stack	<code>std::stack<T></code> (adapts <code>deque</code> by default)
queue	<code>std::queue<T></code>
deque	<code>std::deque<T></code>
doubly linked list	<code>std::list<T></code> (with a sentinel, exactly as in A2)
singly linked list	<code>std::forward_list<T></code>
dynamic array	<code>std::vector<T></code>

`std::list` is a **circular doubly linked list with a sentinel node** in both `libstdc++` and `libc++` — the A2 design, shipped. Its `end()` iterator is the sentinel, which is why `--l.end()` gives you the last element.

7. Why `vector` still usually wins

A `list<int>` node costs 8 bytes of payload and 16 bytes of pointers, is separately allocated, and lands anywhere in memory. A `vector<int>` is one contiguous block. Traversing a million-element `list` can be an order of magnitude slower than the equivalent `vector` because of cache misses and allocator overhead — even though both are $\Theta(n)$. **Reach for a linked list only when you need $O(1)$ splice/erase at a position you already hold**, which is exactly the LRU-cache situation.

Appendix — C++ for Every Pseudocode Block

```
// Shared node type for both list implementations. `explicit` on
the
// single-argument constructor stops an int from silently
converting into a Node
// [Weiss 1.4.2, p.13].
struct Node {
    int key;
    Node* prev = nullptr;      // default member initialisers: no
uninitialised pointers
    Node* next = nullptr;
    explicit Node(int value) : key(value) {}
};
```

A1 LIST-SEARCH, LIST-PREPEND, LIST-INSERT, LIST-DELETE

Pseudocode: §5, "The core operations".

```
class PlainList {
public:
    PlainList() = default;

    // THE BIG-FIVE (toolkit 2). This class owns raw pointers, so
the compiler's
    // defaults are wrong and all five must be stated.
    ~PlainList() { Node* node = head_; while (node) { Node* next =
node->next; delete node; node = next; } }
    PlainList(const PlainList&) = delete;    // no
shallow copy / double free
    PlainList& operator=(const PlainList&) = delete;
    PlainList(PlainList&& other) noexcept : head_(other.head_) {
other.head_ = nullptr; }
    //
    ^^^^^^^^^^^^^^^^^ leaving the
    // source empty is REQUIRED: otherwise both destructors free
the same nodes.
    PlainList& operator=(PlainList&& other) noexcept {
        if (this != &other) { this->~PlainList(); head_ =
other.head_; other.head_ = nullptr; }
```

```

        return *this;
    }

    long long comparisons = 0;          // instrumentation: comparisons
    performed

    // LIST-SEARCH(L, k) -- Theta(n)
    Node* search(int value) {
        Node* node = head_;              // 1  x =
    L.head
        // TWO comparisons per iteration: "am I off the end?" and
        "is this it?".
        // That is exactly what the sentinel version in A2 removes.
        while (node != nullptr && node->key != value) {          // 2
    while x != NIL and x.key != k
            comparisons += 2;
            node = node->next;              // 3
    x = x.next
        }
        if (node != nullptr) comparisons += 2; else comparisons +=
    1;
        return node;                      // 4  return
    x (nullptr if absent)
    }

    // LIST-PREPEND(L, x) -- O(1)
    Node* prepend(int value) {
        Node* node = new Node(value);
        node->next = head_;                // 1  x.next
    = L.head
        node->prev = nullptr;              //   x.prev
    = NIL
        if (head_ != nullptr) head_->prev = node;          // 3  if
    L.head != NIL: L.head.prev = x
        head_ = node;                      // 5  L.head
    = x
        return node;                      // hand the
    pointer back: O(1) erase
    }                                  // is only
    possible if you KEEP it

    // LIST-INSERT(x, y) -- splice x in immediately after y. O(1).

```

```

void insertAfter(Node* node, Node* after) {
    node->next = after->next;           // 1
    node->prev = after;
    if (after->next != nullptr) after->next->prev = node; // 3
    after->next = node;                 // 5
    // ORDER MATTERS: y->next is read into x->next BEFORE being
    overwritten.
    // Swap lines 1 and 5 and you lose the rest of the list.
}

// LIST-DELETE(L, x) -- O(1) GIVEN THE POINTER.
void erase(Node* node) {
    if (node->prev != nullptr) node->prev->next = node->next;
// 1
    else head_ = node->next;           // 3
x was the head
    if (node->next != nullptr) node->next->prev = node->prev;
// 4
    delete node; // the pseudocode stops here; C++ makes you
free the node,
// and forgetting this is the leak Weiss warns
about
}

vector<int> toVector() const {
    vector<int> v;
    for (Node* node = head_; node; node = node->next)
v.push_back(node->key);
    return v;
}

Node* head() const { return head_; }
private:
    Node* head_ = nullptr;
};

```

Complexity. SEARCH $\Theta(n)$; PREPEND, INSERT, DELETE $O(1)$.

The asterisk on **DELETE's** $O(1)$ is the whole lesson of the section: it is $O(1)$ *given a pointer to the node*. Delete-by-key is $\Theta(n)$, because you must search first. Every real use of a linked list — LRU caches above all — pairs it with a hash map from key to node pointer precisely to skip that search.

Verified: 300 randomized runs of 200 mixed prepend / search / erase operations each, cross-checked against a reference `deque`, under `-fsanitize=address,undefined` — no leaks, no invalid reads. Deleting 25 000 nodes given their pointers took 1 198 μ s, i.e. constant time each.

A2 LIST-SEARCH', LIST-DELETE', LIST-INSERT' (the sentinel versions)

Pseudocode: §5, "Sentinels".

```
class SentinelList {
public:
    // The empty list is the sentinel pointing at itself. From this
    // moment on
    // there is NO nullptr anywhere in the structure -- which is
    // what removes
    // every boundary test below.
    SentinelList() { nil_ = new Node(0); nil_>next = nil_; nil_>prev = nil_; }

    ~SentinelList() {
        Node* node = nil_>next;
        while (node != nil_) { Node* next = node->next; delete
node; node = next; }
        delete nil_; // and the sentinel itself
    }
    SentinelList(const SentinelList&) = delete;
    SentinelList& operator=(const SentinelList&) = delete;

    long long comparisons = 0;

    // LIST-INSERT'(x, y) -- no NIL test at all.
    Node* insertAfter(int value, Node* after) {
        Node* node = new Node(value);
        node->next = after->next; // 1
        node->prev = after;
        after->next->prev = node; // 3 y->next is
NEVER null: it is at worst nil_
        after->next = node;
        return node;
    }
}
```

```

    // "No separate procedure for prepending is necessary": head
insertion is
    // insertion after the sentinel, tail insertion is insertion
after nil_->prev.
    Node* prepend(int value) { return insertAfter(value, nil_); }
    Node* append(int value) { return insertAfter(value, nil_-
>prev); }

    // LIST-DELETE'(x) -- TWO lines, versus four with the NIL
checks.
    void erase(Node* node) {
        node->prev->next = node->next;          // 1
        node->next->prev = node->prev;          // 2
        delete node;
    }

    // LIST-SEARCH'(L, k) -- the sentinel-as-guard trick.
    Node* search(int value) {
        nil_->key = value;                      // 1  plant the key in
the sentinel, so the
                                                //    loop is GUARANTEED to
terminate
        Node* node = nil_->next;                // 2
        while (node->key != value) {             // 3  ONE comparison
per iteration, not two
            ++comparisons;
            node = node->next;
        }
        ++comparisons;
        if (node == nil_) return nullptr;      // 5  we only "found" it
in the sentinel
        return node;                          // 7
    }

    vector<int> toVector() const {
        vector<int> v;
        for (Node* node = nil_->next; node != nil_; node = node-
>next) v.push_back(node->key);
        return v;
    }
    Node* nil() const { return nil_; }
private:

```



```
Node* nil_ = nullptr;
};
```

Complexity. Identical to $\mathcal{A}1 - \mathcal{O}(n)$ search, $\mathcal{O}(1)$ insert and delete. **Sentinels change the constant, never the exponent.**

What they actually buy: `erase` drops from four lines with two branches to two lines with none; `insertAfter` loses its null test; `prepend` and `append` stop being special cases; and search does **one** comparison per node instead of two.

CLRS's own verdict, which is worth taking seriously:

*Sentinels often simplify code and might speed up code by a small constant factor, but they **don't typically improve the asymptotic running time**. Use them judiciously. When there are many small lists, the extra storage used by their sentinels can represent **significant wasted memory**.*

One sentinel per list is nothing; one sentinel per list across a million tiny lists is a million wasted nodes.

*Verified: 300 randomized runs against a reference `deque`, clean under ASan/UBSan. On a failed search over $n = 200\,000$ — the worst case, a full traversal — the plain list performed **400 001** comparisons and the sentinel list **200 001**: a ratio of **exactly 2.00**, which is precisely the "two tests per iteration versus one" claim, measured.*

Where you have already used this without knowing: `std::list` in both `libstdc++` and `libc++` is this design. Its `end()` iterator is the sentinel node, which is why `--l.end()` is the last element and why `l.erase(it)` is $\mathcal{O}(1)$ and invalidates nothing else.

Previous: [M05 — Sorting & Order Statistics](#) · Next: [M07 — Hashing](#)